Day 1 — Monolith vs Microservices

> **Activity packet** for your quad. Today's job: frame the monolith-vs-microservices conversation in **business and operational terms** — not framework hype — and write the architecture stance section of your TCD.

Where we are in the week

Week 4 opens by asking: *given the PRD's NFRs, where does this feature belong in the system?* The answer is a **stance**, not a final architectural decision. The architect makes the call; the TPM names the constraints that should drive it.

Inputs from prior weeks:

- Your quad's locked PRD (Week 3 deliverable)
- The NFRs in PRD Section 7
- The integration / dependency list in PRD Section 10

Output today: **Section 1 of the TCD — Architecture stance**, plus a draft of **Section 2 — Integration map** (refined Wednesday with the C4 diagram). Tomorrow's threat model becomes **Section 3**.

The vocabulary card (today's reference)

Term	What it means	TPM-relevant signal
Monolith	One deployable unit; one repo; one runtime	Faster start; harder to scale teams
Modular monolith	Monolith with strict internal module boundaries	Best of both for many cases; underrated
Microservice	Independently deployable, owned by one team	Independent scaling and failure domains; high ops cost
Service-oriented (SOA)	Coarse-grained services; often shared infra	Often what "microservices" actually means in practice
Distributed monolith	Multiple deployables that *must* deploy together	Worst of both — anti-pattern
Function-as-a-service	Per-function deploys (e.g., Azure Functions)	Bursty workloads; not a default
Event-driven	Components communicate via events / queues	Decouples timing; complicates debugging

You need the vocabulary to follow the architect's reasoning — not to win arguments by quoting taxonomy.

The three-question frame (today's mental model)

When someone proposes "let's build it as a microservice", a TPM should be able to ask three questions:

1. Whose deployment cadence does this protect?

Microservices are right when **another team's deployment cadence** would block this team's. If only one team owns the work, a separate service mostly buys ops cost.

2. What independent failure domain do we want?

A separate service is justified when failures should be isolated — e.g., a payments service that should keep processing even if the recommendations service goes down. If the feature *can't function* without its dependencies anyway, separation buys no resilience.

3. What scaling axis are we anticipating?

If 90% of load lives on 10% of features and that 10% has a different scale curve than the rest, separation lets you scale them independently. If load is uniform, one runtime is cheaper.

If at least two of the three answers are "yes, with evidence" — microservice is plausible. If two are "we don't know" — start with a modular monolith.

The "majestic monolith" position

Most early-stage features ship better as **modules in a well-factored monolith** than as new services. The argument:

- **Faster end-to-end testing** (one process, one deploy)
- **Lower observability complexity** (no inter-service tracing tax)
- **Lower coordination cost** (no shared API contract)
- **Reversibility** — you can extract a module into a service later; it's hard to merge two services back

The TPM's job is to **surface the business cost** of premature service-orientation, not to win the debate. Today's stance is a starting point for the architect, not a verdict.

Activity 1 — Mono/Micro Triage

Format: Quad • **45 min** • Block 1

Purpose

Calibrate the room on the three-question frame using public-company examples before applying it to your PRD feature.

Setup

Each quad receives the 8-card triage pack and a stamp sheet for the four stances (Monolith / Microservice / Modular monolith / Need more info).

The triage pack (8 examples, anonymized)

For each, apply the three-question frame and stamp **Monolith fits / Microservice fits / Modular monolith fits / Need more info**:

1. A startup's checkout flow at 50 customers, 1 backend team
2. A streaming service's recommendation engine at scale, 3 teams contributing
3. A new "saved searches" feature in a 2-year-old e-commerce monolith, owned by one squad
4. A payments processor that signs off on transactions for 14 internal apps
5. A reporting export that takes 3–10 minutes per report, run nightly
6. A real-time dispatcher pricing engine that all dispatch flows depend on
7. A small-team's first internal tool, built next to the main app
8. A feature whose load triples every World Cup but is otherwise small

Quad protocol

1. Stamp each card (25 min)
2. Pick the **2 cards with the strongest case** for separation, and the **2 with the strongest case for staying together** (10 min)
3. Identify **1 card** where your quad disagreed (10 min); be ready to explain the disagreement

Readout (60 sec per quad)

> "The clearest case for separation was [X] because [reason]. The clearest case for staying together was [Y] because [reason]. We disagreed about [Z] — the unresolved question is [...]."

Deliverable

Stamped triage pack with reasoning for the two strongest "separate" cases, two strongest "stay together" cases, and one named disagreement.

Activity 2 — Apply the Frame to Your PRD Feature

Format: Quad • **50 min** • Block 2

Purpose

Run the three-question frame against your quad's locked PRD feature and draft Section 1 of the TCD.

Setup

Each quad needs their locked PRD (especially Section 10 Dependencies) and the TCD Section 1 template. AI optional but governed by Week-2 Day-4 provenance rules.

Quad protocol

1. **Re-read PRD Section 10 (Dependencies)** (5 min). Which systems does the feature touch?
2. **Three-question pass** (25 min). For each of the three questions, write a 2-sentence honest answer with evidence. "We don't know" is allowed and informative.
3. **Draft the stance** (15 min). One paragraph in the TCD Section 1 form:

```
## 1. Architecture stance

We propose this feature ship as **<modular monolith
/ new module
in the existing service / new microservice /
hybrid>** because:

- **Deploy cadence:** [one sentence — whose tempo
are we protecting]
- **Failure domain:** [one sentence — what should
keep running if X fails]
- **Scaling axis:** [one sentence — what curve do we
expect]

**Trade-off accepted:** [the cost of this choice —
be honest]
**Revisit if:** [the trigger that would change the
answer]
```

4. **Sanity check** (5 min). Does the stance reference real evidence (deploy frequency, traffic patterns, team boundaries) or hand-wave?

What "good" looks like

- The stance is **defended in business terms** (deploys, failures, scale) — not "modern architecture says…"
- Trade-offs are named, not hidden ("we accept higher coupling for faster ship")
- The "revisit if" is concrete — a metric or organizational change, not "later"

Deliverable

TCD Section 1 (Architecture stance) drafted: stance, three-question answers with evidence, named trade-off, and a concrete revisit trigger.

Activity 3 — Integration Map (first pass)

Format: Quad • **50 min** • Block 3

Purpose

Begin Section 2 of the TCD by listing the systems your feature depends on or extends. Wednesday will draw the diagram; today we list and characterize.

Setup

Each quad needs PRD Section 10 (Dependencies) and the integration table template with the five columns (System, Owner, Sync/async, R/W, Failure handling).

The integration table

For each integration, capture:

System / service	Owner team	Synchronous or async?	Read / Write / Both	Failure handling
Auth (SSO)	Identity	Sync	Read	Fail open / fail closed?
Audit event store	Platform	Async	Write	Drop or queue?
Tickets API	Dispatch	Sync	Both	Retry policy?
Notification service	Comms	Async	Write	Drop / queue / DLQ?

The **Failure handling** column forces a choice the team often defers — make it explicit now.

Quad protocol

1. **List integrations** (25 min). Draw from PRD Section 10 plus anything you forgot.
2. **Characterize each** (15 min). Sync/async, read/write, failure mode.
3. **Identify the "two-way contracts"** (10 min). Which integrations require coordination with another team? Mark these — they go on the dependency owner list.

Deliverable

TCD Section 2 first-pass integration table with sync/async, R/W, and named failure-handling stance per integration. Two-way contracts flagged.

Activity 4 — AI-Assisted Architecture Q&A (Reintroducing AI)

Format: Quad • **55 min** + Wrap • Block 4

Purpose

Reintroduce AI as a research and structuring assistant — without surrendering judgment. The Week 1 prompt patterns and Week 2 Day 4 provenance discipline both apply.

Setup

Each quad needs the TCD Section 1 draft + integration table, the two Pattern Library prompts, and a fresh provenance log entry. AI is allowed and required.

The two AI-assisted exercises

A. "Stress-test our stance"

Use the **Critique-hat prompt** from your Pattern Library:

```
Role: Senior staff engineer reviewing an
architecture stance.
Context: <paste TCD Section 1 + integration table>
Task: Identify the 3 strongest objections to the
stance.
Constraints:
  - Treat the proposed approach as a starting point,
not a verdict
  - For each objection, name the specific scenario
where it would matter
  - Flag any place the rationale relies on
assumptions not stated above
Format: Numbered list of objections, each with a
scenario.
```

B. "What did we forget?"

```
Role: TPM coaching another TPM.
Context: <paste integration table>
Task: For a feature with this integration footprint,
what categories of
       dependency are commonly forgotten? List 5.
Constraints:
  - Be specific to the systems already listed; do
not invent new ones
  - Flag any answer that is generic SaaS boilerplate
Format: Numbered list, each item naming a category
and a specific check.
```

Quad protocol

1. **Run Prompt A** (20 min). Capture the 3 objections.
2. **Decide which objections to adopt** (10 min). For each: adopt / defer / reject — same discipline as Week 3.
3. **Run Prompt B** (10 min). Use the answers to add 1–2 missing integrations to the table.
4. **Update Section 1 + Section 2 with provenance** (10 min). Add a small "AI use" note: which prompts, what was adopted, what was rejected.
5. **Sanity check** (5 min). Does any of this sound like AI-generic prose? Rewrite in your own voice.

Deliverable

Updated TCD Sections 1–2 incorporating adopted AI objections and added integrations, plus an AI-use note logging prompts, adoptions, and rejections.

Wrap (last 15 min)

Each quad shares:

- Their architecture stance (one sentence)
- The strongest objection AI surfaced — and whether they accepted it
- One open architecture question for tomorrow's threat-modeling work

---

End-of-day checkpoint

Each quad ends Day 1 with:

- [x] Section 1 of the TCD — Architecture stance, with trade-off and revisit trigger
- [x] Section 2 (first pass) — Integration table with failure-handling stances
- [x] An AI-use note (provenance log) — what prompts were used, what was adopted/rejected
- [x] One open question for the architect or security partner